

Sapienza

Università di Roma

Ingegneria dell'Informazione, Informatica e
Statistica

Laurea Triennale

Dipartimento di Informatica

Basi di Dati

Appunti a cura di
Lorenzo Ferraro

27 settembre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it>

Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Basi di Dati*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale github.com/TonnoArcobaleno/uni-notes, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

Segnalazioni

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@gmail.com**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

Indice

1	Introduzione alle basi di dati	1
1.1	Sistema informativo	1
1.2	Gestione dei dati	1
1.2.1	Informazioni condivise	3
1.3	Sistemi informatici	3
1.3.1	Sistemi informatici operativi	4
1.3.2	Sistemi informatici di supporto alle decisioni	4
1.3.3	OLTP vs OLAP	4
1.3.4	Requisiti per l'analisi dei dati	5
1.4	Big Data e Data Lake	5
1.5	Modelli di DBMS	6
1.5.1	Prima del modello relazionale	6
1.5.2	Modello relazionale	6
1.5.3	Linguaggio per gestire i DB	7
1.5.4	Concorrenza	9
1.6	Progettare una base di dati	10
1.6.1	Problemi della progettazione di DB	12
1.6.2	Modellare la realtà	12
1.6.3	Associazioni	16

Capitolo 1

Introduzione alle basi di dati

1.1 Sistema informativo

Definizione 1.1 – Sistema informativo

Un **sistema informativo** di un'organizzazione è una combinazione di risorse, umane e materiali, e di procedure organizzate per *raccogliere, archiviare, elaborare e scambiare* le informazioni necessarie alle attività **operative** (informazioni di servizio), di **programmazione e controllo** (informazioni di gestione) e di **pianificazione strategica** (informazioni di governo).

Questo sistema è quindi una componente dell'organizzazione che viene utilizzata per gestire le informazioni di interesse, supportando le altre componenti dell'organizzazione (come il sistema produttivo o decisionale, ad esempio).

Definizione 1.2 – Sistema informatico

Il **sistema informatico** è l'insieme delle tecnologie informatiche e della comunicazione (ICT) a supporto delle attività di un'organizzazione.

Definizione 1.3 – Sistema informativo automatizzato

Il **sistema informativo automatizzato** è quella parte del sistema informativo dove le informazioni sono raccolte, elaborate, archiviate e scambiate tramite l'ausilio di un sistema informatico.

Dato che ormai quasi tutti i sistemi informativi sono digitalizzati, i tre termini appena visti tendono spesso a sovrapporsi e a essere usati come sinonimi.

1.2 Gestione dei dati

Negli anni 60'/70' ogni applicazione aveva un proprio file privato dove venivano salvati i propri dati. Questo creava una serie di problemi:

- **ridondanza**, dato che se due applicazioni usavano gli stessi dati questi venivano salvati due volte.
- **inconsistenza**, dato che aggiornare un dato in un file non significava aggiornarlo automaticamente in tutti i file
- **dipendenza dei dati**, dato che ogni applicazione organizzava i propri dati tenendo conto solamente dell'uso che doveva farne

Proprio per questo motivo nascono le **basi di dati**.

Definizione 1.4 – Basi di Dati

Una **Base di Dati** (o Database, **DB**) è un **insieme di file connessi tra loro**. Gli insiemi di dati sono organizzati in delle *strutture di dati* che rendono più facile la creazione, l'accesso e l'aggiornamento di tali e ottimizzano la gestione delle risorse fisiche.

Definizione 1.5 – Dati

I **dati** sono dei fatti grezzi che bisogna interpretare e correlare per fornire una vera informazione.

Ad esempio, 3336669990 da solo è un semplice numero, ma se è la risposta alla domanda "Qual è il numero di telefono di X", allora ha tutto un altro significato, e diventa quindi una *informazione* vera e propria.

Quando si registrano delle informazioni, questi possono essere

- **dati strutturati**, ossia rappresentati da brevi stringhe di simboli e da numeri,
- o **dati non strutturati**, ossia testi scritti in linguaggio naturale.

La struttura dell'informazione dipende dal suo utilizzo e può essere modificata. Ad esempio, un tempo bastava il nome e il cognome di una persona per identificarla in modo efficace, mentre oggi utilizziamo anche la data di nascita, il luogo di nascita e, se necessario, il codice fiscale.

L'obiettivo di una base di dati non è solo il memorizzare i dati in modo compatto, ma soprattutto **facilitarne l'elaborazione** sulla base delle loro relazioni. Ad esempio, parlando dei dati strutturati, vi si può accedere singolarmente tramite delle "interrogazioni", dette *query*, e le relazioni tra i dati individuali sono salvate nella *struttura dei record*.

I dati contenuti DB sono suddivisi in due categorie:

- **dati**: rappresentazioni di fatti conformi alla definizione dello schema delle basi di dati. Questi hanno diverse caratteristiche:
 - Sono organizzati in insieme strutturati e omogenei, con relazioni definite tra i vari insiemi.

- Sono molti di più rispetto ai metadati, e non sono gestiti nella RAM (memoria temporanea).
 - Sono accessibili tramite transazioni, sequenze di istruzioni che devono tutte andare a buon fine per apportare vere modifiche.
 - Sono protetti sia dall'accesso di utenti non autorizzati, sia da malfunzionamento di hardware e/o software.
 - Sono utilizzabili contemporaneamente da più utenti.
- **metadati**: descrivono fatti sullo schema dei dati, utenti autorizzati, applicazioni e così via. Sono descritti da uno schema e sono interrogabili come i dati.

Definizione 1.6 – Sistemi di Gestione di Basi di Dati

I **Sistemi di Gestione di Basi di Dati** (o Database Management System, **DBMS**) sono **software per la gestione di molti dati** che si trovano nella memoria secondaria.

Questi offrono gli strumenti per **definire lo schema** di un DB, che va definito prima di inserire i dati, per **scegliere le strutture dati adeguate**, per **memorizzare i dati** rispettando i vincoli dello schema scelto, e per **recuperare/modificare** i dati in modo interattivo o attraverso dei programmi. Sono dunque il ponte tra gli utenti e il DB vero e proprio.

1.2.1 Informazioni condivise

In un'organizzazione, ogni componente è interessata a una porzione del Sistema Informativo. Quando queste porzioni si sovrappongono, la base di dati diventa fondamentale, facendo da *struttura condivisa* tra tutte le componenti, riducendo le ridondanze e le inconsistenze. Questa condivisione, però, non è sempre completa. Infatti, non tutte le componenti dell'organizzazione possono accedere a tutti i dati di questa: le informazioni più private devono essere salvaguardate. Inoltre, la condivisione comporta la necessità di gestire accessi in contemporanea sugli stessi dati: questo è il problema del **controllo della concorrenza**.

1.3 Sistemi informatici

I sistemi informatici si dividono in due macrogruppi:

- **sistemi informatici operativi**
- **sistemi informatici di supporto alle decisioni** (direzionali)

1.3.1 Sistemi informatici operativi

I **sistemi informatici operativi** (o Data processing, DP / Electronic Data processing, EDP / Transaction Processing Systems, TPS) sono organizzati in basi di dati. Qui le applicazioni sono usate per svolgere le attività strutturate e ripetitive di un'azienda nell'area amministrativa e finanziaria. Abbiamo quindi una base di dati, gestita da un DBMS, che lavora con più applicazioni in ambiti come la vendita, la contabilità, la finanza...

Definizione 1.7 – OLTP

L'**OLTP** (On-Line Transaction Processing) è il **paradigma architetturale** usato dai DBMS nei sistemi informatici operativi. Le sue operazioni sono predefinite e relativamente semplici, e ognuna di queste coinvolge pochi dati, che sono però dettagliati e aggiornati.

1.3.2 Sistemi informatici di supporto alle decisioni

I dati nei **sistemi informatici di supporto alle decisioni** (Decision Support Systems, DSS / Management Information Systems, MIS / Executive Information System, EIS) sono organizzati in **Data Warehouse** (DW) e gestiti da un sistema apposito. Le applicazioni, dette di *Business Intelligence*, supportano i processi di controllo delle prestazioni aziendali e di decisione manageriale. Abbiamo quindi una base di dati operativa e dei possibili dati esterni che vanno dentro il DW che viene gestito da un DBMS dedicato, e infine il sistema OLAP (che ora vedremo) interroga il DW e fornisce le informazioni necessarie agli strumenti di Business Intelligence.

Definizione 1.8 – OLAP

L'**OLAP** (On-Line Analytical Processing) è il **paradigma** usato nei DSS per analizzare i dati. Le sue operazioni sono complesse e casuali, e ognuna coinvolge molti dati, che sono aggregati, storici e non sempre molto attuali.

1.3.3 OLTP vs OLAP

Vediamo le molte differenze tra OLTP e OLAP.

- L'OLTP supporta l'**operatività** dell'azienda, mentre l'OLAP le sue **decisioni**
- L'OLTP ha **molti utenti**, mentre l'OLAP è più **esclusivo**
- I dati dell'OLTP sono **analitici e relazionali**, mentre quelli dell'OLAP sono **sintetici e multidirezionali**
- Gli utilizzi dell'OLTP sono **noti a priori**, mentre quelli dell'OLAP **non sono molto prevedibili**.

- Per attività, la quantità di dati dell'OLTP è **bassa** (si parla di decine di dati), mentre quella dell'OLAP è **alta** (si parla di milioni di dati)
- L'OLTP è **orientato all'applicazione**, mentre l'OLAP al **soggetto**
- Nell'OLTP gli aggiornamenti sono **frequenti**, mentre nell'OLAP sono **rari**
- Nell'OLTP i dati sono **sovrascritti** (i dati vecchi non sono interessanti), mentre nell'OLAP sono **importanti anche i dati vecchi**
- L'OLTP è ottimizzato per le **transazioni**, mentre l'OLAP per l'**analisi dei dati**

1.3.4 Requisiti per l'analisi dei dati

I dati dell'OLAP sono presi per essere analizzati. Questi hanno bisogno di requisiti specifici:

- Devono essere **aggregati**: non ci interessa il singolo dato, ma un calcolo su più dati, come la somma, la media o la mediana.
- Le informazioni devono essere **incrociate**: bisogna spesso analizzare i dati da punti di vista diversi.
- Bisogna analizzare i dati su **diversi livelli di dettaglio**: ad esempio, se si scopre un calo nelle vendite in un periodo di una regione, bisogna analizzare quei dati in modo più approfondito per capirne le cause

I sistemi informatici operativi, essendo progettati per un'elaborazione rapida delle transazioni, sono dunque l'**perfetti per OLTP** ma non adatti per l'OLAP.

1.4 Big Data e Data Lake

Definizione 1.9 – Big Data

Big Data è un termine ampio che si riferisce a tutte le situazioni dove l'approccio *schema-first* tipico dei DB e delle DW è troppo lento o restrittivo. Si parla in questo caso delle 3 V:

- **Volume**
- **Varietà**
- **Velocità**

Sono questi 3 fattori a mandare in crisi i sistemi tradizionali (quantità di dati o eccessive, o troppo variegate, o dati che devono essere elaborati velocemente).

I Big Data sono associati solitamente ai sistemi NoSQL, al Machine Learning, e all'**approccio Data Lake**.

Definizione 1.10 – Data Lake

Un **Data Lake** è un **ambiente di data storage** a basso costo creato per gestire enormi quantità di dati non elaborati e di qualsiasi formati, inclusi dati strutturati, semi-strutturati e non strutturati.

Quasi tutti i Data Lake utilizzano lo storage a oggetti basato su cloud, come Google Cloud Storage. Questi sono nati per gestire il flusso di nuovi big data, molti non strutturati, creati da app e servizi connessi a Internet. A differenza dei DB e DW tradizionali, questi non richiedono uno schema di dati definito e possono anche memorizzare diversi tipi di dati in più formati tutti nello stesso posto. Questo li rende più ideali per i data scientist, grazie al suo schema *on-read*: i dati vengono strutturati solo quando vanno letti, al contrario dello schema *on-write* delle DW dove i dati sono strutturati fin da subito.

1.5 Modelli di DBMS

1.5.1 Prima del modello relazionale

Il modello relazionale è ad oggi lo standard per i DBMS. Lo analizzeremo tra poco, ma per ora diamo un'occhiata ai modelli che lo hanno preceduto.

Il primo modello di DB è il **modello gerarchico**, uscito a metà degli anni 60'. Questo organizza i dati in una struttura ad albero dove ogni record figlio può avere solo un genitore. Questo modello è stato poi esteso dal modello **reticolare**, che ha trasformato l'albero (che è un grafo aciccolare e connesso) in un grafo generico.

A metà degli anni 80' arrivano anche i primi sistemi **orientati a oggetti**. In questo modello troviamo gli *attributi* che descrivono lo stato di un oggetto, e i *metodi* che ne descrivono invece il comportamento. Al contrario del modello relazionale, però, non esiste ancora un modello universalmente riconosciuto.

1.5.2 Modello relazionale

Definizione 1.11 – Modello relazionale

Nel 1970 E.F. Codd, lavorando per IBM, ideò il **modello relazionale**, che diventerà lo standard nel 1993. Qui **dati e relazioni sono rappresentati entrambi come valori**. Non ci sono puntatori espliciti come nei modelli reticolari e gerarchici, portando l'astrazione ad un livello più in alto.

I record sono gli **oggetti**, mentre le informazioni di interesse sono i **campi**. Lavoriamo con delle **tabelle** di oggetti omogenei, dove le colonne sono rappresentate dai campi mentre le righe dagli oggetti appunto. Vediamo un esempio di DB relazionale:

Studenti			Corsi		
Matricola	Nome	Cognome	Codice	Titolo	Insegnante
123456	Mario	Rossi	01	Analisi	Aiello
654321	Andrea	Pirlo	02	Chimica	Sterbini
456789	Francesco	Totti	03	Fisica	Carlucci
987654	Cristiano	Ronaldo			

Esami		
Studente	Voto	Corso
123456	21	02
987654	30	03
456789	28	01

Notiamo come le relazioni tra i diversi oggetti siano rappresentate dai valori stessi appunto: nella tabella Esami, riusciamo a capire chi ha svolto quale esame e ha preso quale voto grazie ai valori nella tabella. Ad esempio, capiamo che Mario Rossi ha preso 21 al corso di Chimia del prof. Sterbini. È importante capire, però, che queste relazioni non dipendono dal nome del campo: bisogna sempre poter rinominare le colonne senza perdere le relazioni.

1.5.3 Linguaggio per gestire i DB

Ora che abbiamo definito cosa sono i DB, abbiamo bisogno di un **linguaggio** per gestirli. Per capire come definire questo linguaggio, però, dobbiamo prima capire appieno come sono strutturati i database. Possiamo distinguere 3 livelli distinti:

- **Vista Logica**, ossia le diverse viste degli utenti in base ai loro diritti di accesso. Non tutti possono accedere a tutti i dati del database, e per questo è necessario creare viste diverse.
- **Logico**, ossia la descrizione strutturale del database, come i dati vanno suddivisi, senza però importarsi di come questi vengano salvati fisicamente.
- **Fisico**, ossia l'implementazione fisica dei dati sulle memorie di massa.

Per progettare un database, bisogna partire dal livello **logico**. Dobbiamo qui capire quali dati vogliamo salvare nel nostro DB, le loro relazioni, i loro vincoli e in che modo strutturare le tabelle. Riprendendo l'esempio visto prima con le tabelle Studenti, Corsi e Esami, avremmo scritto:

```
Studenti(Matricola char(8), Nome char(20), Cognome char(20))
Corsi(Codice char(2), Titolo char(20), Insegnante char(20))
Esami(Studente char(8), Voto int, Corso int)
```

Si passa poi al livello **fisico**, che descrive quali strutture dati ausiliarie bisogna utilizzare per facilitare l'uso del DB. Infine, vi sono le **viste logiche**, che permettono ad utenti con vari livelli di accesso di vedere solo determinate informazioni.

Proposizione 1.1 – Indipendenza fisica e logica

Tramite l'approccio con tre livelli, si garantisce l'**indipendenza fisica e logica** dei dati nel DBMS. Infatti, i programmi non devono essere modificati a seguito di modifiche sull'organizzazione fisica (**indipendenza fisica**), e nemmeno a seguito di modifiche allo schema logico (**indipendenza logica**).

Proprietà 1.1 – Modalità di accesso al DBMS

Dato che non tutti sono in grado di accedere a un DBMS come lo farebbe un programmatore, è necessario creare **diverse modalità di accesso** per soddisfare tutti gli utenti.

- **Interfaccia grafica:** è lo strumento visivo per gli utenti finali che fa interagire al DBMS solo tramite bottoni e tabelle. Il più semplice di tutti.
- **Linguaggio di interrogazione per utenti non programmatori:** sono quelle interfacce usate dagli utenti per cercare qualcosa con dei filtri selezionati. L'esempio perfetto è quando si cerca su Amazon un oggetto con dei filtri selezionati (ad esempio prezzo < 50€).
- **Linguaggio di programmazione per sviluppatori di applicazioni:** sono quei linguaggi che fanno comunicare l'applicazione con il DBMS, come ad esempio Java o Python.
- **Linguaggio per lo sviluppo di interfacce di applicazioni:** sono quei linguaggi usati per progettare la grafica attraverso cui l'utente finale vede le tabelle e legge/inserisce i dati.

Proprietà 1.2 – Proprietà dei DB

Un DBMS deve garantire le seguenti proprietà di una base di dati:

- **Integrità**, ossia mantenere i dati coerenti con i vincoli prestabiliti.
- **Sicurezza**, ossia proteggere i dati da usi non autorizzati.
- **Affidabilità**, ossia proteggere i dati da malfunzionamenti hardware e software.

Riepiloghiamo quindi i vantaggi e gli svantaggi dei DBMS:

Tabella 1.1: Vantaggi e Svantaggi di un DBMS

Vantaggi	Svantaggi
Indipendenza dei dati	Necessario schema logico
Recupero efficiente dei dati	Possono essere costosi e complessi
Integrità e sicurezza dei dati.	Dati solo strutturati e omogenei
Dati protetti da malfunzionamenti	Non adatti per le applicazioni OLAP
Accessi concorrenti gestiti correttamente	
Amministrazione dei dati	
Riduzione dei tempi per lo sviluppo di applicazioni	
Ottimi per le applicazioni OLTP	

Definizione 1.12 – Transazioni

Una **transazione** è una **sequenza di azioni** di lettura e scrittura in memoria permanente e di elaborazioni di dati in RAM, che segue una serie di proprietà chiamate **ACID**.

Le proprietà ACID sono:

- **Atomicità:** le transazioni che per qualche motivo terminano prima di essere completate sono eliminate totalmente, come se non fossero mai iniziate. Dunque deve essere completata completamente (*committed*), oppure non deve essere eseguita affatto (*rolled back*).
- **Consistenza:** le transazioni devono lasciare i dati in uno stato coerente con i vincoli imposti.
- **Isolamento:** in caso di più transazioni, gli effetti di una non devono dipendere dalle altre.
- **Durabilità (Persistenza):** le modifiche fatte da una transazione terminata sono permanenti.

Un esempio di transazione è: "Trasferisci 1000€ dal conto c1 al conto c2". Il DBMS in questo caso cerca c1, modifica il suo saldo in saldo - 1000, cerca c2 e modifica il suo saldo in saldo + 1000.

1.5.4 Concorrenza

Quando due transazioni che lavorano sugli stessi dati lavorano insieme, possono verificarsi dei problemi. Facciamo un esempio.

Ipotizziamo che in un DBMS due utenti in facciano partire due transazioni: la transazione 1 è "Aggiungi 1000€ sul conto c", mentre la transazione 2 è "Aggiungi 500€ sul conto c". Le transazioni partono insieme, e nel DBMS succede questo:

1° la transazione 1 cerca c1

2° la transazione 2 cerca c1

3° la transazione 1 modifica il saldo di c1 in saldo + 1000

4° la transazione 2 modifica il saldo di c1 in saldo + 500

Partendo con un valore saldo = 0, ci si aspetterebbe che il saldo finale sia saldo = 1500, ma in realtà il valore sarà saldo = 500. Questo perché ogni transazione, una volta letto un valore (in questo caso il saldo di c1), lo salva in un suo spazio di memoria privato. Dunque in questo caso la prima transazione salva il valore 0, e anche la seconda. La prima quindi aggiorna il saldo al valore 1000, ma la seconda lo sovrascrive a 500.

Si potrebbe pensare di risolvere questo problema semplicemente verificando a posteriori che il risultato sia corretto, ma questo è praticamente impossibile: infatti, in un DBMS reale arrivano transazioni di continuo, e dunque per fare queste verifiche dovremmo perdere un sacco di tempo e annullare molte transazioni mentre verifichiamo che sia tutto ok.

Proposizione 1.2 – Transazioni concorrenti

Per risolvere il problema della concorrenza, bisogna far sì che più transazioni in parallelo **lavorino come se fossero in serie**. Esistono per questo scopo dei **protocolli** con regole rigide a priori che tutte le transazioni devono rispettare.

1.6 Progettare una base di dati

Per progettare una base di dati si intende progettare la struttura dei dati e delle applicazioni. Questa è la fase più importante e delicata, ma non è l'unica: fa parte di un contesto più grande, ossia il **ciclo di vita dei sistemi informativi**.

Definizione 1.13 – Ciclo di vita dei sistemi informativi

Il **ciclo di vita dei sistemi informativi** è la sequenza di attività svolte da analisti, progettisti, utenti... che permette di sviluppare e usare una base di dati.

Prende il nome di ciclo perché una fase può essere ripetuta anche più volte prima di arrivare al risultato finale, magari per risolvere dei problemi che si sono presentati solo dopo averla conclusa. Questo ciclo è composto da 6 fasi:

- 1° **Studio della fattibilità**: si valuta se il progetto è effettivamente fattibile con le risorse disponibili.
- 2° **Raccolta e analisi dei requisiti**: si decide cosa deve fare il sistema nel concreto chiedendo agli utenti quali funzioni sono necessarie e quali no (spesso eseguita in contemporanea con la fase di Progettazione).
- 3° **Progettazione**: si traducono le richieste degli utenti in modelli astratti e schemi formali.

- 4° **Implementazione:** si scrive il codice, si creano le tabelle e le si iniziano a popolare con i dati iniziali.
- 5° **Validazione e collaudo:** si verifica che il sistema rispetti tutti i requisiti e vincoli e che non ci siano errori.
- 6° **Funzionamento:** si utilizza il sistema normalmente, monitorandolo costantemente per verificare il corretto funzionamento.

Questo ciclo di vita dei sistemi informativi si traduce per le basi di dati in 4 fasi:

- 1° **Analisi dei requisiti**
- 2° **Progettazione concettuale**
- 3° **Progettazione logica**
- 4° **Progettazione fisica**

Analizzeremo più avanti ogni singola fase, ma è importante capire che la progettazione si divide sempre in due aspetti:

- **Progettazione di dati**
- **Progettazione di applicazioni**

Entrambe le progettazioni, però, avvengono attraverso lo stesso mezzo: i **modelli informatici**.

Definizione 1.14 – Modello informatico

Un **modello informatico** (o, più in generale, **modello astratto**) è la rappresentazione formale di idee e di conoscenze relative a un fenomeno.

La realtà è enorme e disordinata: per questo usiamo dei modelli che selezionano solo alcune conoscenze utili a descrivere il fenomeno che ci interessa, tramite costrutti matematici o schemi fissi. La stessa realtà, quindi, in base alle necessità può essere modellata in più modi, conservando ogni volta informazioni diverse.

Proprietà 1.3 – Tipi di modelli

Esistono due tipi di modelli:

- **modelli logici:** indipendenti dalle strutture fisiche ma supportati nativamente dai DBMS, come ad esempio il modello relazionale.
- **modelli concettuali:** indipendenti dalle modalità di realizzazione, rappresentanti le entità del mondo reale e le loro relazioni nelle prime fasi della progettazione del DB, come ad esempio il modello ER (Entità-Relazione) che studieremo nello specifico più avanti. Si trovano a un livello di astrazione ancora più alto dei modelli logici.

Definizione 1.15 – Modello di dati

Un **modello di dati** è un insieme di strutture utilizzate per organizzare i dati di interesse e le loro relazioni. Il componente fondamentale in queste strutture sono i **costruttori di tipo**, ossia un meccanismo che permette di combinare tipi semplici per formare tipi complessi.

Per capire cosa sono i costruttori di tipo facciamo un esempio. Dei tipi semplici per un oggetto di tipo *Studente* possono essere ad esempio *Matricola* (testo), *Nome* (testo) e *Cognome* (testo). Se prendiamo un altro tipo semplice *Voto* (intero), combinando *Matricola* e *Voto* possiamo creare un *Record* (*Matricola*, *Voto*) che rappresenta un esame. Questo ci è possibile farlo proprio grazie al costruttore di tipo.

1.6.1 Problemi della progettazione di DB

Durante la progettazione di una base di dati vi sono diversi aspetti problematici da analizzare:

- 1) **Aspetto ontologico**: cosa bisogna rappresentare della realtà nel nostro DB?
- 2) **Aspetto logico**: come modelliamo i dati?
- 3) **Aspetto linguistico**: che linguaggio formale usiamo?
- 4) **Aspetto pragmatico**: per chi stiamo creando questa base di dati? Di cosa necessitano i nostri utenti?

Parlando dell'aspetto ontologico, bisogna distinguere tre tipi di conoscenza:

- Conoscenza *concreta*, ossia i fatti da rappresentare
- Conoscenza *astratta*, ossia i vincoli sulla conoscenza concreta
- Conoscenza *procedurale*, ossia le operazioni che gli utenti potranno effettuare sui dati

Ci concentreremo in futuro in particolare sui primi due tipi di conoscenza.

Per l'aspetto logico invece ci concentriamo sullo studio dei modelli di dati, che sono dei meccanismi per descrivere la struttura della conoscenza concreta. Questo definisce le classi, gli oggetti, le relazioni... usati per descrivere la realtà, e definisce anche lo schema usato. Per il momento useremo una notazione che prende il nome di **diagramma ER (Entità-Relazione)**.

1.6.2 Modellare la realtà

Definizione 1.16 – Entità

Un'entità rappresenta un **elemento del mondo reale** di cui ci interessa rappresentare alcune **proprietà**. Ogni entità appartiene a un **tipo**, che ne specifica la natura, e può far parte di una **collezione**, ossia un insieme di entità omogenee variabile nel tempo.

Definizione 1.17 – Proprietà di un'entità

Le **proprietà** sono fatti che ci interessano solo perché **descrivono caratteristiche di determinate entità**. Queste sono rappresentate da una coppia di valori del tipo (Attributo, valore di un certo tipo).

N.B. Alcuni fatti in un contesto potrebbero essere interpretati come proprietà e in altri come entità. Ad esempio, un Autore in una Biblioteca può essere una proprietà di un'entità Libro, ma può essere un'entità a parte per una Casa Editrice.

Ad esempio, Diego è un'entità di tipo Persona, con proprietà (Nome, "Diego":string) e (Età, 20:int), che fa parte della Collezione delle persone che studiano alla Sapienza.

Definizione 1.18 – Oggetto

Un **oggetto** è la definizione informatica formale di un'entità del dominio, caratterizzato da 3 aspetti:

- **stato**, ossia l'insieme delle costanti e variabili che lo modellano
- **comportamento**, ossia l'insieme delle sue procedure locali che prendono il nome di *metodi*
- **identità**, che lo rende unico e indistinguibile

Un oggetto può poi rispondere a una serie di messaggi attraverso valori memorizzati o calcolati al momento. Un **tipo oggetto** è l'insieme dei messaggi a cui un insieme di oggetti può rispondere. I nomi di questi messaggi sono detti **attributi degli oggetti**.

Definizione 1.19 – Classe

Una **classe** è un **insieme di oggetti** dello stesso tipo, modificabile in modo da includere o escludere elementi specifici dall'insieme. È la controparte informatica della collezione di entità.

Una stessa classe può essere rappresentata a diversi livelli di specifica. Ad esempio, la stessa classe Persona può essere rappresentata in tre modi differenti:

Persona	Persona	Persona
	Nome	Nome: string
	Cognome	Cognome: string
	DataNascita	DataNascita: date
	Sesso	Sesso: (M; F)
	Indirizzo	Indirizzo: [Via: string; Citta: string]
	LingueParlate	LingueParlate: seq string

Definizione 1.20 – Gerarchie di classi

Spesso le classi sono organizzate in una **gerarchia**, dove vi è una classe più generale detta **superclasse** e una o più classi più specifiche dette **sottoclassi**. Esistono le **relazioni di sottoclasse**, con le seguenti proprietà:

- Sono *antisimmetriche, riflessive e transitive*
- Se C è sottoclasse di C' , allora il tipo degli elementi di C è sottotipo degli elementi di C' (*vincolo intensionale*)
- Se C è sottoclasse di C' , allora gli elementi di C sono un sottoinsieme degli elementi di C' (*vincolo estensionale*)

Nelle gerarchie le proprietà sono **ereditate**, e gli elementi che compongono una sottoclasse sono un sottoinsieme degli elementi della superclasse. Ad esempio, in ambito universitario gli Studenti sono una superclasse, mentre gli Studenti di Ingegneria e gli Studenti di Medicina sono due sottoclassi.

Definizione 1.21 – Gerarchia tra tipi oggetto

Oltre alle gerarchie di classe vi sono anche le **gerarchie di tipi oggetto**. Esistono anche qui le **relazioni di sottotipo**, con diverse proprietà:

- Sono *antisimmetriche, riflessive e transitive*
- Se T è sottotipo di T' , allora gli elementi di T possono essere usati in ogni contesto in cui sono usati gli elementi di T' (*principio di sostitutività*)

Definizione 1.22 – Ereditarietà

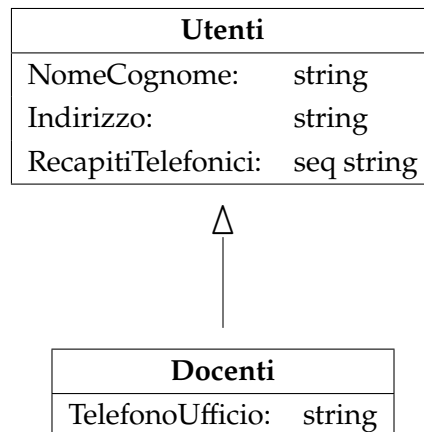
Abbiamo già definito il concetto di **ereditarietà** con le gerarchie di classi e di tipo oggetto, ma vi sono due diversi tipi di ereditarietà:

- **Ereditarietà di tipo**, che definisce solo quali attributi e metodi deve avere la classe derivata
- **Ereditarietà di implementazione**, che rappresenta concretamente il codice della classe derivata

Si parla poi di **ereditarietà stretta** quando il codice l'ereditarietà viene usata non solo per riciclare codice, ma per creare delle vere sottoclassi logiche. Qui gli attributi delle sottoclassi ereditati possono solo essere aggiunti, e non cancellati (non si possono perdere attributi delle superclassi, così da mantenere il principio di sostitutività), ma si possono ridefinire solo specializzandone il tipo.

Per indicare l'ereditarietà tra classi usiamo una freccia che va dalla sottoclasse alla superclasse che si chiude con un triangolo vuoto. Ad esempio, per definire Docenti come una sottoclasse di

Utenti si usa uno schema di questo tipo:



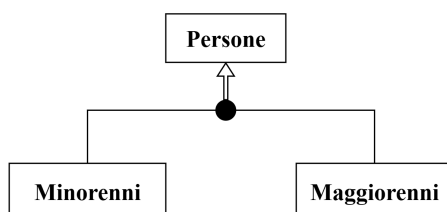
Proprietà 1.4 – Proprietà sottoclassi

Quando una superclasse viene suddivisa in più sottoclassi, si definiscono due proprietà:

- **Disgiunzione** (Esclusività): le sottoclassi sono esclusive se gli elementi di una non fanno parte di nessun'altra
- **Copertura**: le sottoclassi sono totali se l'unione di tutte queste copre l'intera popolazione della superclasse

Se entrambe le proprietà sono rispettate, le sottoclassi formano una **partizione** della superclasse.

Per dire che le sottoclassi sono disgiunte si usa un pallino nero, mentre per dire che coprono la superclasse si usa una doppia freccia.



In questo esempio, Minorenni e Maggiorenni sono due sottoclassi esclusive e coprono anche tutte le persone, dunque c'è sia il pallino che la doppia freccia.

1.6.3 Associazioni

Definizione 1.23 – Associazioni

Un'associazione è un fatto che correla due o più entità, creando un legame tra loro.

Un'associazione $R(X, Y)$ fra due collezioni di entità X e Y è un insieme di istanze di associazione tra gli elementi di X e di Y . Il prodotto cartesiano $X \times Y$ è detto **dominio dell'associazione**. Le associazioni possono avere poi anche delle proprie **proprietà**, e possono essere anche **ricorsive** (o riflessive).

N.B. Il concetto di associazione riprende in modo chiaro il concetto di **relazione matematica**.

Prendiamo per esempio due insiemi di entità: l'insieme Autori = {"Pirandello", "Leopardi"} e l'insieme Testi = {"Uno, Nessuno, Centomila", "La Ginestra"}. Un'istanza di associazione sarebbe ad esempio la coppia ("Pirandello", "Uno, Nessuno, Centomila"), mentre il dominio dell'associazione sarebbe l'insieme $\text{Autori} \times \text{Testi} = \{("Pirandello", "Uno, Nessuno, Centomila"), ("Pirandello", "La Ginestra"), ("Leopardi", "Uno, Nessuno, Centomila"), ("Leopardi", "La Ginestra")\}$.

Proprietà 1.5 – Vincolo di molteplicità

Un'associazione $R(X, Y)$ è **univoca rispetto a X** se per ogni elemento x di X esiste al più un elemento y di Y associato a x . Se questo vincolo non è rispettato, allora l'associazione è **multivalore rispetto a X**.

- $R(X, Y)$ è **(1:N)** se è *multivalore su X* e *univoca su Y*. Infatti, preso un elemento y di Y , esiste solo una corrispondenza in X , mentre preso un elemento x in X esistono più corrispondenze in Y .
- $R(X, Y)$ è **(N:1)** se è *univoca su X* e *multivalore su Y*.
- $R(X, Y)$ è **(N:M)** se è *multivalore su X* e *multivalore su Y*.
- $R(X, Y)$ è **(1:1)** se è *univoca su X* e *univoca su Y*.

Ad esempio, l'associazione Frequenta(Studenti, Corsi) ha **cardinalità (N:M)**. Infatti, uno studente può frequentare più corsi e un corso può essere frequentato da più studenti, mentre l'associazione NatoA(Persone, Città) ha cardinalità (N:1), dato che una persona può nascere in una sola città, ma in una città possono nascere più persone.

Proprietà 1.6 – Vincolo di totalità

Un'associazione $R(X, Y)$ è **totale su X** se per ogni elemento x di X esiste almeno un elemento y di Y associato a x . Se questo vincolo non è rispettato, allora l'associazione è **parziale su X**.

Ad esempio, l'associazione Frequenta(Studenti, Corsi) è totale su Corsi, dato che non può esistere un corso non frequentato da nessuno (verrebbe cancellato), ed è totale anche su Studenti (non esiste studente che non partecipi a nessun corso: in tal caso non sarebbe uno studente). Mentre l'associazione NatoA(Persone, Città) è totale su Persone, dato che ogni persona è nata per forza in una città, ma parziale su Città, dato che può esistere una città dove non è nato nessuno.